Frontispiece

TRAINING MANUAL

Coding With //Screen-Readers

CONFIDENTIAL — For Internal Use Only

Copyright © 2026 Michael Ryan Hunsaker, M.Ed., Ph.D. Coding With Screen-Readers is licensed under a Creative Commons Attribution-NoNCommerical-NoDerivatives 4.0 International: https://creativecommons.org/licenses/by-nc-nd/4.0/ All rights reserved. NO AI TRAINING: Without in any way limiting the author's rights under copyright, any use of this publication to "train" generative artificial intelligence (AI) technologies to generate text is expressly prohibited.

Contents

Teacher's Guide: Introduction to Computer Science with Java 3
Unit 0: Setting Up Your Java Development Environment 30
Unit 1: Foundations of Computing and Getting Started with Java 32
Unit 2: Writing Simple Programs and Input/Output 38
Unit 3: Control Flow — Decisions and Loops 40
Unit 4: Data Structures — Arrays and ArrayLists 45
Unit 5: Strings and String Processing 49
Unit 6: Methods (Functions) 51
Unit 7: Classes and Object-Oriented Programming 53
Unit 8: Files and Exception Handling 57
Unit 9: Collections, Data Analysis, and Testing 63
Unit 10: Project-Based Learning 68
Unit 11: Capstone Project — Personal Data Manager in Java 73

Dedication

To the countless individuals who have been told they lack the capability to code, yet remain determined to learn and create through accessible coding practices, this book is dedicated.

Epigraph

The power of the Web is in its universality. Access by everyone regardless of disability is an essential aspect

— Tim Berners-Lee, Inventor of the World Wide Web

Introduction to Computer Science with Java

For Visually Impaired Students

Course Overview

This curriculum introduces computer science concepts using Java, adapted for students who rely on screen readers. All exercises are text-based and terminal-focused. No graphical output is required.

Target Audience: Students with no prior programming experience using screen readers (NVDA, JAWS, VoiceOver, or Windows Narrator).

Tools Required

- **Java Development Kit (JDK) 17+** — `https://adoptium.net/`
- **Visual Studio Code** with the **Extension Pack for Java** by Microsoft
- A screen reader (NVDA, JAWS, or VoiceOver)

Accessibility Notes

- All program output is text-based (console/terminal).
- VS Code's Screen Reader Optimized Mode works well with Java.
- Configure your screen reader to announce indentation changes — Java uses braces `{}` to define blocks, but consistent indentation is still essential for readability.
- Use `Ctrl + Shift + P` -> "Toggle Screen Reader Optimized" in VS Code.
- Use `F8` / `Shift+F8` to navigate between errors.

Units

Unit	Title
Unit 0	Setting Up Your Java Development Environment
Unit 1	Foundations of Computing and Getting Started with Java
Unit 2	Writing Simple Programs and Numeric Computations

Unit	Title
Unit 3	Control Flow — Decisions and Loops
Unit 4	Data Structures — Arrays and ArrayLists
Unit 5	User Input and String Processing
Unit 6	Methods (Functions)
Unit 7	Classes and Object-Oriented Programming
Unit 8	Files and Exception Handling
Unit 9	Collections and Basic Data Analysis
Unit 10	Project-Based Learning
Unit 11	Capstone Project

References

- Eck, D.J. (2022). *Introduction to Programming Using Java* (9th ed.). Free PDF available at https://math.hws.edu/javanotes/
- Horstmann, C.S. (2018). *Big Java: Early Objects* (6th ed.). Wiley.

Teacher's Guide: Introduction to Computer Science with Java

For Students with Visual Impairments — Complete Instructor Edition

---

Table of Contents

1. Philosophy and Why Java
2. Java-Specific Accessibility Challenges
3. Classroom Setup and Configuration
4. Spoken Java Conventions
5. Unit 0–1: Setup and Foundations
6. Unit 2: I/O and the Math Class
7. Unit 3: Control Flow
8. Unit 4: Arrays and ArrayLists
9. Unit 5: Strings and String Processing
10. Unit 6: Methods
11. Unit 7: Classes and OOP
12. Unit 8: Files and Exception Handling
13. Unit 9: Collections, Analysis, and Testing
14. Units 10–11: Projects and Capstone
15. Bridging from Python to Java
16. Assessment Strategies
17. Supporting Diverse Needs
18. Alternative and Supplementary Resources
19. Quick Reference Sheets

---

1. Philosophy and Why Java

The Case for Teaching Java to Blind Students Java is not the easiest language for beginners, and its verbosity creates overhead that requires specific pedagogical attention. Despite this, Java has genuine advantages for visually impaired learners that are often overlooked:

Compile-time error detection. Java's compiler catches type errors, name errors, and structural mistakes before any code runs. A blind student does not need to search through runtime output for subtle type mismatches — the compiler announces them precisely and immediately with file name, line number, and error description.

Strong, static typing is auditorily informative. When IntelliSense in VS Code suggests methods, it includes the return type: "`double getBalance()` — returns a double." This type information helps a screen reader user understand what a method does without having to navigate to its definition.

Enforced code structure. Every Java program requires a class; every executable program requires a `main` method with a specific signature.

This structure is more overhead to type but less ambiguity about how programs are organized. For a student who cannot visually scan a file for structure, predictable structural conventions reduce cognitive load.

Exception specification. Java's checked exceptions force programmers to declare which exceptions a method can throw. This means a screen reader user reading a method signature gets information about failure modes that Python provides only through documentation.

Enterprise relevance. Java remains a dominant language in enterprise software, Android development, and many backend systems. A student who learns Java learns patterns that transfer directly to professional employment.

---

2. Java-Specific Accessibility Challenges

The Verbosity Problem Java requires significantly more text than Python to accomplish the same task:

```java
// Java: Hello, World
public class HelloWorld {
    public static void main(String[] args) {
        System.out.println("Hello, World!");
    }
}
```

```python
# Python: Hello, World
print("Hello, World!")
```

For a sighted student, extra text is a visual annoyance. For a blind student, it is additional audio before they reach the meaningful code. **Mitigation strategies:**

1. **Code templates.** Provide a starter file the student copies for every new program. They modify only the method body for early lessons. This reduces the typing burden and prevents structural errors while the student is learning concepts.

2. **JShell (Java REPL).** Java includes an interactive shell since Java 9: run `jshell` in the terminal. JShell strips away class and method boilerplate. For exploration and experimentation, JShell is far more accessible than creating files:

```
jshell> int x = 5;
x ==> 5
jshell> System.out.println(x * 2);
10
```

3. **Progressive disclosure.** In early lessons, treat `public static void main(String[] args) {` as a formula, not something to understand. Students learn to recognize it by ear. Explain each word only when the concept it relates to (access modifiers, static methods, arrays) is being taught.

Integer Division Surprises Java's `/` operator performs integer division when both operands are integers:

- `15 / 4` = `3` (not `3.75`)
- `15.0 / 4` = `3.75`
- `(double) 15 / 4` = `3.75`

This is silent — there is no error or warning. A blind student may not catch a calculation that silently discards its decimal portion. Teach the rule explicitly: **if you want a decimal result, at least one operand must be a `double`.**

The String Equality Trap `==` on Strings compares memory addresses, not content. Two strings with identical characters in different memory locations are `==` false but `.equals()` true. This never appears in Python; blind students coming from Python are not conditioned to suspect `==` as a source of String bugs.

Teaching approach: Show the broken behavior explicitly. Use `new String("hello") == new String("hello")` which is guaranteed to be `false` even though the strings are identical. Then show `.equals()` which is `true`. Make `.equals()` for Strings a non-negotiable rule from day one.

Null Pointer Exceptions `NullPointerException` is the most common Java runtime error. It occurs when you call a method or access a field on a variable that holds `null` (no object reference). Unlike Python's `AttributeError` which gives useful context, Java's NPE message is often terse. Teaching students to recognize and debug NPEs is essential.

---

3. Classroom Setup and Configuration

IDE Configuration **VS Code with Extension Pack for Java:** This is the recommended setup. The Extension Pack includes Language Support for Java (Red Hat), Debugger for Java, Java Test Runner, Maven for Java, and Project Manager for Java.

Alternative IDEs to consider:

IDE	Accessibility	Notes
VS Code + Extension Pack	High	Recommended. Full accessibility via keyboard.
IntelliJ IDEA Community	Medium-High	Powerful but complex. More keyboard shortcuts to learn. JetBrains has an accessibility guide.
Eclipse	Medium	Older UI; can be navigated but more complex accessibility model.
BlueJ	Low–Medium	Designed for teaching; visual design features may impede accessibility.
DrJava	Low–Medium	Simple; not actively maintained.
jGRASP	Low	Visualization-focused; primarily visual.
Greenfoot	Low	Visual/game-focused; not accessible for screen reader use.

VS Code Java Specific Settings:

```
{
  "java.completion.enabled": true,
  "java.format.enabled": true,
  "editor.formatOnSave": true,
  "java.saveActions.organizeImports": true,
  "java.completion.importOrder": ["java", "javax", "com", ""],
  "editor.suggestOnTriggerCharacters": true
}
```

Key accessibility setting: Enable "Trigger suggestions" on `.` so that typing `myObject.` causes IntelliSense to announce available methods. This allows audio exploration of an object's API.

```
java --version      # Runtime version
javac --version     # Compiler version
jshell              # Start REPL (Ctrl+D or /exit to quit)
```

JDK Installation Verification All three must work before proceeding. If `jshell` is not found, the student may have installed only the JRE (runtime), not the full JDK. The JDK must be installed.

---

4. Spoken Java Conventions

Extend the Python symbol conventions with Java-specific additions:

Symbol/Pattern	Spoken Form
`public`	"public"
`private`	"private"
`static`	"static"
`void`	"void" (returns nothing)
`new`	"new"
`this.`	"this dot"
`super.`	"super dot"
`@Override`	"at-override annotation"
`// comment`	"double-slash comment"
`/* ... */`	"block comment"
`/** ... */`	"JavaDoc comment"
`String[]`	"String array"
`ArrayList<String>`	"ArrayList of String"
`HashMap<String, Integer>`	"HashMap from String to Integer"
`throws IOException`	"throws IOException"
`System.out.println()`	"sysout" or "system-out-println"
`System.out.printf()`	"system-out-printf"
`Integer.parseInt()`	"Integer-parse-int"
`(int) x`	"cast to int"
`null`	"null"
`NullPointerException`	"null pointer exception"
`ArrayIndexOutOfBoundsException`	"array index out of bounds"

Reading a method signature:

```
public double calculateArea(double radius) throws IllegalArgumentException
```

"public double method calculateArea, takes a double called radius, may throw IllegalArgumentException"

---

5. Unit 0–1: Setup and Foundations

Lesson 0.1: Installing the JDK

Instructional Cues

- **Use Adoptium (Eclipse Temurin) rather than Oracle JDK.** The Oracle JDK requires account creation and license agreement navigation. Adoptium is fully open-source and simpler to download.
- **Verify the download before running.** Adoptium provides SHA checksums. Teaching students to verify downloads is a security habit worth establishing.

- **Windows installer tip:** The Adoptium `.msi` installer includes a checkbox for "Set JAVA_HOME variable." Ensure this is checked. Many Java tools (Maven, Gradle, some IDEs) require `JAVA_HOME` to be set.

JShell as the First Environment Before any file-based Java, spend time in JShell:

```
$ jshell
|  Welcome to JShell -- Version 17.0.x
|  For an introduction type: /help intro

jshell> 5 + 3
$1 ==> 8

jshell> "Hello " + "World"
$2 ==> "Hello World"

jshell> int x = 42;
x ==> 42

jshell> System.out.println("Value: " + x);
Value: 42
```

JShell gives immediate auditory feedback — type an expression, hear the result. This is much more accessible for early exploration than the compile-run-read cycle.

JShell commands to teach:

- `/list` — show all code entered so far
- `/edit` — open in editor
- `/vars` — show all variables
- `/methods` — show all methods defined
- `/exit` — quit JShell

Thought Experiments

1. *"Python is interpreted — each line runs immediately. Java compiles to bytecode first, then runs. What does this mean for the kind of errors you encounter, and when? If you have a SyntaxError on line 50, in Python you might not discover it until the program reaches line 50. In Java, when do you discover it?"*

2. *"The Java Virtual Machine (JVM) allows the same Java bytecode to run on Windows, macOS, and Linux. The JVM is different on each platform, but the bytecode is the same. What does this tell you about how Java achieves portability? What alternative approaches could a language designer take?"*

3. *"JShell is a Read-Eval-Print Loop (REPL). Python has one; JavaScript has one; many languages have one. But Java didn't have one until 2016. Why might a language's designers add a REPL 20 years after the language was created? What did programmers do without it?"*

4. *"Java was originally designed for embedded systems (like televisions), then became dominant for web servers, then for Android apps. Each time it succeeded in a new domain, it required extending the language. What does this history suggest about the relationship between a language's design and the problems it eventually solves?"*

Lesson 1.2: Your First Java Program

Instructional Cues **Anatomy reading exercise — before typing any code:** Read the Hello World program aloud, symbol by symbol, using the pronunciation table:

```
public class HelloWorld {
    public static void main(String[] args) {
        System.out.println("Hello, World!");
    }
}
```

"public class HelloWorld open-brace newline four-spaces public static void main open-paren String array args close-paren open-brace newline eight-spaces System dot out dot println open-paren quote Hello comma World! close-quote close-paren semicolon newline four-spaces close-brace newline close-brace"

This exercise establishes pattern recognition for the boilerplate that the student will type hundreds of times. Making it auditorily familiar early reduces cognitive load later.

Three `System.out` methods:

```
System.out.println("With newline at end");     // Most common
System.out.print("No newline at end");          // For prompts
System.out.printf("Pi is %.2f%n", Math.PI);    // Formatted
```

For accessible programs, `printf` is extremely valuable — it produces consistent, predictable output formatting that reads cleanly with a screen reader.

Thought Experiments

1. *"Every Java program requires a `public class` whose name matches the file name. Why does Java enforce this one-to-one correspondence? What would break if you could put multiple public classes in one file?"*

2. *"The `main` method signature is exactly `public static void main(String[] args)`. It cannot be `public void main()` or `static void main(String[] args)`. Why is this signature fixed? What does each word contribute?"*

3. *"Java uses curly braces `{}` to delimit code blocks; Python uses indentation. From an accessibility perspective — for a blind programmer — which approach provides better information about structure when read aloud? Consider that braces can be read as 'open block' and 'close block,' while indentation changes require screen reader configuration."*

4. *"`System.out.println()` is long to type but descriptively named. Python's `print()` is short. What naming philosophy does each represent? When does conciseness become ambiguity?"*

Lesson 1.3: Variables and Primitive Data Types

Instructional Cues **Type declaration as specification:** Frame Java's type declarations positively: *"When you write `int score = 85`, you're telling Java: this variable will always hold a whole number. Java then protects you — if you accidentally try to put a name or a decimal into it, the compiler tells you immediately."*

The eight primitive types — prioritize four: In this curriculum, students need: `int`, `double`, `boolean`, `char`. The others (`byte`, `short`, `long`, `float`) exist but are rarely needed in beginner programs. Introduce the four primary types fully before acknowledging the others exist.

Type casting demonstration:

```
// Widening (automatic)
int i = 100;
double d = i;   // 100 -> 100.0, automatic

// Narrowing (explicit, may lose data)
double pi = 3.14159;
int truncated = (int) pi;  // 3.14159 -> 3, data LOST

System.out.println("Truncated: " + truncated);  // 3, not 4!
```

The truncation behavior (not rounding) is consistently surprising. Demonstrate it with `(int) 3.9` producing `3`, and ask: "Is this rounding or truncating? What is the difference? When would truncation cause a bug?"

Non-Obvious Accessibility Strategies

- **`System.out.printf()` for consistent numeric output:** `printf("Balance: $%.2f%n", balance)` always produces exactly 2 decimal places. Without formatting, `double` values may produce varying numbers of decimal places (`3.0`, `3.14`, `3.1415926535...`) that read differently each time. Consistent formatting makes output more predictable.

- **`Scanner.hasNext___()` methods:** Before calling `scanner.nextInt()`, check `scanner.hasNextInt()`. If the user typed a non-integer, this returns `false` and prevents the exception, allowing graceful re-prompting. For a blind student who may not immediately catch an exception in terminal output, this is defensive programming with real accessibility value.

Thought Experiments

1. *"Java's `int` holds values from about -2.1 billion to +2.1 billion. What happens if you exceed this limit (integer overflow)? The value silently wraps around to the opposite extreme. Name a real-world system where integer overflow has caused a documented failure."*

2. *"Java has `int` (primitive) and `Integer` (object). They hold the same values but behave differently. `int` cannot be null; `Integer` can. `int` cannot be used in a `List<int>` — you need `List<Integer>`. Why would a language designer make this distinction? What does it cost? What does it protect against?"*

3. *"Java's `char` holds a single Unicode character. `char c = 'A'` stores the letter A. But `char` is also a numeric type — `(int) 'A'` is `65`. What does this dual nature (character and number) tell you about how computers store text internally?"*

---

6. Unit 2: I/O and the Math Class

Lesson 2.2: Scanner for User Input

Instructional Cues **The Scanner as a wrapper — explain the layers:**

```
Keyboard
  V produces bytes
System.in (InputStream)
  V raw byte stream
new Scanner(System.in)
  V wraps with text-reading methods
scanner.nextLine(), scanner.nextInt(), etc.
```

This layered model explains why `Scanner` is needed rather than reading from `System.in` directly. It also foreshadows file reading where `new Scanner(new File(...))` replaces `new Scanner(System.in)`.

`nextInt()` / `nextLine()` pitfall — mandatory demonstration:

```
Scanner scanner = new Scanner(System.in);
System.out.print("Enter your age: ");
int age = scanner.nextInt();          // Reads "25", leaves "\n" in buffer
System.out.print("Enter your name: ");
String name = scanner.nextLine();     // Reads the leftover "\n" --- gets EMPTY string!
System.out.println("Hello " + name + ", age " + age);
// Output: "Hello , age 25" --- name is empty!
```

The fix:

```
int age = scanner.nextInt();
scanner.nextLine();          // Consume the leftover newline
String name = scanner.nextLine();   // Now reads properly
```

Run the broken version first. The student should hear the incorrect output and understand why before seeing the fix.

Thought Experiments

1. *"The `Scanner` reads from a 'stream' of input. A stream is a sequence of data that flows from a source to a destination. `System.in` is the keyboard stream. What other streams exist in a typical program? (Think about file streams, network streams, the output stream.)"*

2. *"After `scanner.nextInt()`, there's a leftover newline in the buffer. What IS a buffer? Why does the operating system buffer keyboard input rather than delivering each character the moment a key is pressed?"*

3. *"In GUI (graphical) programs, users fill out forms with text fields and click buttons. There's no `scanner.nextLine()`. What mechanism do GUI programs use to collect user input? How does event-driven GUI input differ from sequential console input?"*

---

7. Unit 3: Control Flow

Lessons 3.1–3.3

Instructional Cues **String comparison — immediate practice:**

```
String day = "Wednesday";

// Wrong --- compares memory addresses
```

```
if (day == "Wednesday") {
    System.out.println("Middle of the week");  // May or may not print
}

// Correct --- compares content
if (day.equals("Wednesday")) {
    System.out.println("Middle of the week");  // Always correct
}

// Case-insensitive
if (day.equalsIgnoreCase("wednesday")) {
    System.out.println("This works too");
}
```

Create a practice file with 10 comparisons, half using `==` and half using `.equals()`. Ask the student to predict which ones will work correctly before running.

Switch expressions (Java 14+):

Modern Java supports switch expressions that return a value:

```
String dayType = switch (day) {
    case "Monday", "Tuesday", "Wednesday", "Thursday", "Friday" -> "Weekday";
    case "Saturday", "Sunday" -> "Weekend";
    default -> "Unknown";
};
System.out.println(day + " is a " + dayType);
```

This eliminates the fallthrough problem and is more concise. Teach both traditional `switch` (for reading legacy code) and switch expressions (for writing new code).

Thought Experiments

1. *"Java uses `&&` for AND, `||` for OR, `!` for NOT. Python uses the words `and`, `or`, `not`. Which approach is more readable when heard aloud by a screen reader? Is there a meaningful accessibility argument for one over the other?"*

2. *"The enhanced for loop `for (String item : list)` is simpler to write than the traditional `for (int i = 0; i < list.size(); i++)`. But sometimes you need the index. Name three operations you can do with the traditional loop that you cannot do (directly) with the enhanced loop."*

3. *"Java's `switch` statement traditionally falls through from one case to the next unless you write `break`. This 'feature' has caused countless bugs. Modern Java (14+) has switch expressions that don't fall through. What does this change tell you about how programming language design evolves in response to real-world usage patterns?"*

4. *"Short-circuit evaluation: `if (obj != null && obj.getValue() > 0)` — if `obj` is null, the second condition is never evaluated. This prevents*

a NullPointerException. How does this behavior make certain coding patterns possible that would otherwise crash?"

---

8. Unit 4: Arrays and ArrayLists

Lesson 4.1: Arrays

Instructional Cues **The fixed-size limitation — demonstrate, don't just describe:**

```
String[] fruits = new String[3];
fruits[0] = "apple";
fruits[1] = "banana";
fruits[2] = "cherry";
fruits[3] = "date";   // ArrayIndexOutOfBoundsException!
```

Let the student hear the exception and analyze the message: `Index 3 out of bounds for length 3`. Ask: "The message gives you two numbers. What are they? What does each tell you?"

`Arrays.toString()` as mandatory print method:

The first time `print(fruits)` produces `[Ljava.lang.String;@7a5d012b`, stop and explain: "This is the memory address of the array, not its contents. Java arrays don't have a built-in 'print nicely' behavior. Always use `Arrays.toString(fruits)` or loop through manually."

The `for` vs `for-each` tradeoff — make it concrete:

```
// for-each: simple, clear, but no index
for (String fruit : fruits) {
    System.out.println(fruit);
}

// traditional: more complex, but gives index
for (int i = 0; i < fruits.length; i++) {
    System.out.println(i + ": " + fruits[i]);
}

// The numbered version is more accessible!
```

For blind students, numbered output (showing both index and value) is generally more useful than unnumbered output. Teach the traditional `for` loop for array printing specifically because it enables indexed output.

Thought Experiments

1. *"Arrays are stored in contiguous memory — all elements in a row, one after another. This is why accessing by index is fast: the address of element `i` is*

just the start address plus `i × element size`. Why does this contiguous storage make inserting into the middle of an array slow? What must happen to all the elements after the insertion point?"

2. *"If you create `String[] arr = new String[5]`, what are the initial values of `arr[0]` through `arr[4]`? Now create `int[] nums = new int[5]` — what are the initial values? Why are the defaults different between String arrays and int arrays?"*

3. *"Two-dimensional arrays in Java are 'arrays of arrays.' This means each row can have a different length — a 'jagged array.' When would a jagged array be more appropriate than a rectangular 2D array? Give a real-world data example."*

Lesson 4.2: ArrayLists

Instructional Cues **Autoboxing — explain the invisibility:**

```
ArrayList<Integer> numbers = new ArrayList<>();
numbers.add(42);        // 42 (int) is auto-boxed to Integer(42)
int first = numbers.get(0);  // Integer(42) is auto-unboxed to 42
```

This happens invisibly but has a performance cost. For the course, tell students: "Use `ArrayList<Integer>` when you need a flexible list of integers. The boxing/unboxing is automatic but slightly less efficient than an array of `int`."

Removal by index vs. by value:

```
ArrayList<String> list = new ArrayList<>(Arrays.asList("a", "b", "c", "b", "d"));
list.remove(1);            // Removes by INDEX --- removes "b" at position 1
list.remove("b");          // Removes by VALUE --- removes first "b" found
list.removeAll(Arrays.asList("b", "c"));  // Removes all matching values
```

Ask the student: "If I call `list.remove(0)` on a list of integers `[10, 20, 30]`, does this remove the element at index 0 (which is `10`) or remove the value `0` (which doesn't exist)? How does Java decide?" (Answer: removes index 0, because `0` is an `int`, not an `Integer`.)

Thought Experiments

1. *"An `ArrayList` doubles its internal capacity when it runs out of space. If you start with capacity 16 and keep adding elements, when does reallocation occur? Trace the capacity: 16 -> 32 -> 64 -> 128… What is the total cost (in element moves) of adding 1000 elements to an initially empty ArrayList this way? Is this efficient?"*

2. *"Java generics `ArrayList<String>` enforce type safety at compile time. Python lists can hold any types mixed together. When is a typed list (Java style) better? When is a mixed list (Python style) more flexible? Which approach produces more reliable programs?"*

---

9. Unit 5: Strings and String Processing

Instructional Cues **String immutability — the Java version:**

In Java, Strings are immutable for the same reason as Python. But Java's `String` object is also **interned** — the JVM may reuse the same `String` object for two variables holding the same literal value. This is why `==` sometimes works for String comparison (when both refer to the same interned object) but is not reliable.

`StringBuilder` motivation:

```
// Inefficient --- creates 100,000 String objects
String result = "";
for (int i = 0; i < 100000; i++) {
    result += "x";
}

// Efficient --- modifies a single buffer
StringBuilder sb = new StringBuilder();
for (int i = 0; i < 100000; i++) {
    sb.append("x");
}
String result = sb.toString();
```

The `+` in a loop creates a new String object on every iteration, then discards the old one. For small loops this is fine; for large loops it's a significant performance problem. `StringBuilder` maintains a mutable buffer.

Thought Experiments

1. *"String concatenation with `+` is convenient but expensive in loops. In Python, the equivalent is also discouraged — use `''.join(list)` instead. What does this common advice across two languages tell you about how immutable strings interact with loops?"*

2. *"Java's `String.format()` and `printf()` use format specifiers like `%s`, `%d`, `%f`. Python uses f-strings with `{}`. What are the accessibility tradeoffs of each approach? Which reads more naturally when heard aloud? Which is easier to type?"*

3. *"Regular expressions are a tiny language inside a programming language. `\\d+` matches one or more digits. `[a-zA-Z]+` matches one or more letters.*

Why do programmers use regular expressions despite their difficult-to-read syntax? What would the alternative be for complex pattern matching?"

---

10. Unit 6: Methods

Instructional Cues **Method overloading — auditory navigation:**

```
public static int add(int a, int b) { return a + b; }
public static double add(double a, double b) { return a + b; }
public static int add(int a, int b, int c) { return a + b + c; }
```

When IntelliSense shows overloads in VS Code, pressing `Shift+Alt+F2` (Accessible View) lists them all. Teach students to use this to discover available overloads.

`void` vs. return type — make the distinction tangible:

```
// void: for side effects only
public static void printGreeting(String name) {
    System.out.println("Hello, " + name + "!");
}

// Return type: for computation
public static double calculateTip(double billAmount, double rate) {
    return billAmount * rate;
}

// Using them:
printGreeting("Alice");                    // Standalone call
double tip = calculateTip(50.00, 0.18);    // Must capture return value
System.out.printf("Tip: $%.2f%n", tip);
```

"`void` methods exist to DO something. Return-type methods exist to CALCULATE something and give you the result. Ask: 'Do I need to use the output later in my code?' If yes, it needs a return type."

Thought Experiments

1. *"Java methods can throw exceptions: `public double sqrt(double n) throws IllegalArgumentException`. This signature tells callers that the method might fail in a specific way. Python has no equivalent syntax. What is the programmer's obligation when they see `throws` in a method signature?"*

2. *"Static methods belong to the class, not to any instance: `Math.sqrt(25)` — you don't need a `Math` object. Instance methods belong to an object: `myString.length()`. What is the conceptual difference? When should a method be static vs. instance?"*

3. *"Method parameters in Java are 'pass by value' for primitives and 'pass by reference' for objects. For primitives: the method gets a copy, and modifying the parameter doesn't affect the original. For objects: the method gets the reference, and modifications to the object's fields DO affect the original. Design a test to confirm this behavior for both cases."*

4. _"Recursion: a method that calls itself. `int factorial(int n) { return n == 0 ? 1 : n _ factorial(n-1); }`. Every recursive solution has an iterative equivalent. When is the recursive version clearer? What is the risk of deep recursion (many nested calls)?"*

---

11. Unit 7: Classes and OOP

Extended Instructional Cues **The mandatory `__str__` equivalent in Java — `toString()`:**

Every student-defined class must override `toString()`. Without it, `System.out.println(myObject)` produces an unreadable string. The `@Override` annotation makes this explicit:

```
@Override
public String toString() {
    return String.format("Dog[name=%s, age=%d]", name, age);
}
```

Require this in every class from Unit 7 onward. Explain: "When your screen reader reads `System.out.println(myDog)`, it reads whatever `toString()` returns. If you don't define `toString()`, you'll hear a memory address. That's useless."

JavaDoc comments as accessible documentation:

Teach this format for every method:

```
/**
 * Calculates the area of this circle.
 *
 * @return the area as a double, in square units
 * @throws IllegalArgumentException if radius is negative
 */
public double getArea() {
    if (radius < 0) throw new IllegalArgumentException("Radius cannot be negative");
    return Math.PI * radius * radius;
}
```

When IntelliSense shows this method in VS Code, pressing `Shift+Alt+F2` reads the JavaDoc aloud. This makes the student's own code navigable via audio — a profound accessibility benefit.

The VS Code symbol navigator: `Ctrl+Shift+O` opens a dropdown listing all methods in the current file. Type a few letters to filter. This

allows jumping to any method by name — much more efficient than arrow-key navigation through a long file.

Access modifiers — enforce from day one:

Modifier	Rule	Reason
`private` on fields	Always	Encapsulation; prevents direct manipulation
`public` on getters/setters	When needed	Controlled access
`private` on helper methods	Yes	Internal implementation detail
`public` on methods the user calls	Yes	Interface to the class

Establish this as a non-negotiable habit, not a preference. *"Private fields, public interface. Every time."*

Thought Experiments

1. *"Encapsulation means the internal state of an object is private. Only its public methods can access or modify it. Why would you want to prevent direct access to `account.balance`? What bugs could occur if balance were public?"*

2. *"Inheritance creates a hierarchy. A `SavingsAccount` extends `BankAccount`. Every `SavingsAccount` is-a `BankAccount`, but not every `BankAccount` is-a `SavingsAccount`. What are the type-safety implications of this? If you have a `BankAccount` variable, can it hold a `SavingsAccount` object?"*

3. *"Java interfaces define contracts — a list of methods that any implementing class must provide. A class can implement multiple interfaces but can only extend one class. Why this asymmetry? What problem does single inheritance solve? What does it prevent?"*

4. *"Design Pattern: the Factory Method creates objects without specifying the exact class. Instead of `new Dog()`, you'd call `AnimalFactory.create("dog")`. What is the advantage of this indirection? When would it be useful?"*

5. *"Object-oriented programming was supposed to model the real world. A `Dog` is-a `Animal`. But real-world relationships are complex — a `FlyingFish` is both a `Fish` and can fly like a `Bird`. How does Java's single-inheritance model handle this situation? What alternatives exist?"*

6. *"Polymorphism means one interface, many implementations. `Animal[] animals = {new Dog(), new Cat(), new Bird()}; for (Animal a : animals) { a.speak(); }` — each animal speaks differently. Why is this*

more flexible than checking `instanceof` *and calling different methods explicitly?"*

Alternative Resources

- **"Head First Design Patterns" by Freeman, Robson et al.:** Visual book, but audio-describable concepts. Covers fundamental OOP design patterns. For advanced students.
- **"Effective Java" by Joshua Bloch:** The definitive book on Java best practices. Dense but invaluable for understanding WHY Java is designed the way it is.
- **Oracle Java Tutorials (docs.oracle.com/javase/tutorial/):** Comprehensive, plain HTML, fully accessible. Covers OOP concepts with extensive examples.

---

12. Unit 8: Files and Exception Handling

Instructional Cues **Checked vs. unchecked exceptions — the core distinction:**

Type	Examples	Must Handle?	Why?
Checked	`IOException`, `FileNotFoundException`, `SQLException`	Yes — compiler enforces	These are recoverable; programmer should handle them
Unchecked (RuntimeException)	`NullPointerException`, `ArrayIndexOutOfBoundsException`, `IllegalArgumentException`	No — optional	Usually indicate programming errors; may not be recoverable

"Checked exceptions are like warning labels: 'This operation can fail. You must decide what to do if it does.' Unchecked exceptions are usually bugs — the program did something wrong."

Try-with-resources — teach first, manual close second:

```
// Modern: try-with-resources (always teach this first)
try (BufferedReader reader = new BufferedReader(new FileReader("data.txt"))) {
    String line;
    while ((line = reader.readLine()) != null) {
        System.out.println(line);
    }
```

```
} catch (IOException e) {
    System.out.println("Error reading file: " + e.getMessage());
}
// reader is automatically closed here, even if an exception occurred

// Legacy: manual close (show this so students understand what try-with-resources does)
BufferedReader reader = null;
try {
    reader = new BufferedReader(new FileReader("data.txt"));
    // ... use reader
} catch (IOException e) {
    System.out.println("Error: " + e.getMessage());
} finally {
    if (reader != null) {
        try { reader.close(); } catch (IOException e) { /* ignore */ }
    }
}
```

Show the legacy version to explain why try-with-resources exists and what it simplifies.

Thought Experiments

1. *"Checked exceptions force you to handle failures at compile time. Critics argue this creates 'exception chaining hell' — every method signature becomes littered with `throws` clauses. Defenders argue it makes failure modes explicit and programs more reliable. Which side do you find more persuasive? Why?"*

2. *"Java's `Path` and `Files` classes (java.nio.file) are a modern replacement for the older `File` class. The older `File` class's `delete()` method returns a `boolean` (true/false) instead of throwing an exception. What is the danger of reporting failure via a return value? What might a programmer accidentally do wrong?"*

3. *"A `NullPointerException` is called 'The Billion Dollar Mistake' by its inventor Tony Hoare. Explain why null references cause so many bugs. Newer languages (Kotlin, Swift) have built-in null safety. What would Java code look like if null didn't exist?"*

---

13. Unit 9: Collections, Analysis, and Testing

JUnit Testing Deep Dive

Instructional Cues **Test output is inherently accessible:** JUnit produces plain text output: . for each passing test, `F` for failures, with descriptive failure messages. This is one of the most screen-reader-friendly outputs

in the entire course. Let students experience testing by running a test suite and listening to the results before writing their own tests.

Test naming convention — sentence style:

```
@Test
public void calculateArea_withPositiveRadius_returnsCorrectValue() { }

@Test
public void calculateArea_withZeroRadius_returnsZero() { }

@Test
public void calculateArea_withNegativeRadius_throwsException() { }
```

The naming convention `methodName_condition_expectedBehavior` makes test names read as complete sentences when the test runner announces them. This is both good practice and an accessibility benefit — each test announcement tells you exactly what was tested.

`assertThrows` — test that exceptions work correctly:

```
@Test
public void withdraw_withInsufficientFunds_throwsException() {
    BankAccount account = new BankAccount(100.0);
    assertThrows(InsufficientFundsException.class, () -> account.withdraw(200.0));
}
```

Testing for exceptions is as important as testing for correct outputs. Many beginners only test the "happy path."

Thought Experiments

1. *"Test-driven development (TDD) says: write the test first, then write the implementation. How does writing a test first change the way you think about a method? What is forced to be clear before you write a line of implementation?"*
2. *"Mocking means replacing real dependencies (like a database or file system) with fake versions for testing. Why would you want to test a method that reads from a database without using a real database? What complexity does mocking introduce?"*
3. *"A project has 100% test coverage — tests execute every line. Is the software correct? Name a bug that 100% coverage would not catch. (Hint: think about a bug that only appears with certain combinations of inputs.)"*

---

14. Units 10–11: Projects and Capstone

Java Project Architecture For the capstone, encourage a clean package structure:

```
src/
|--- com/
|   +--- student/
|       +--- datamanager/
|           |--- model/
|           |   +--- Contact.java
|           |--- storage/
|           |   |--- DataManager.java
|           |   +--- JsonStorage.java
|           |--- ui/
|           |   +--- ConsoleUI.java
|           +--- Main.java
test/
+--- com/
    +--- student/
        +--- datamanager/
            |--- model/
            |   +--- ContactTest.java
            +--- storage/
                +--- DataManagerTest.java
```

Navigating packages in VS Code: `Ctrl+Shift+E` for Explorer (shows package tree), `Ctrl+P` to open any file by name, `Ctrl+Shift+O` for symbols within the current file.

IDE-Generated Code Accessibility VS Code and IntelliJ can generate getters, setters, constructors, and toString. Teach these as time-savers, but require students to read and understand all generated code:

- `Ctrl+Shift+P` -> "Java: Generate Getters and Setters"
- `Ctrl+Shift+P` -> "Java: Override/Implement Methods"
- `Ctrl+Shift+P` -> "Java: Generate toString"

Generated code is correct but often not styled for accessibility (missing JavaDoc, generic names). Require students to add JavaDoc and verify `toString()` output is meaningful when heard.

Thought Experiments

1. *"Java in enterprise software is often criticized for having too much 'boilerplate' — code that exists for structural reasons rather than to express logic. How does this boilerplate affect the experience of maintaining a large Java codebase? What tools and patterns does the Java ecosystem provide to reduce it?"*

2. *"Your capstone application stores data in a file. If your team adds a new field to the data class, existing saved files no longer match the new format.*

This is a 'schema migration' problem. How would you design your storage layer to handle this gracefully?"

---

15. Bridging from Python to Java

For courses that teach Python before Java, use these comparison thought experiments to bridge the languages:

Syntax bridge table:

Python	Java	Notes
`print("Hello")`	`System.out.println("Hello");`	
`x = 5`	`int x = 5;`	Java requires type declaration and semicolon
`if condition:`	`if (condition) {`	Parens around condition; brace instead of colon
`elif`	`else if`	Two words in Java
`for item in list:`	`for (Type item : list) {`	Java requires type
`def func(x):`	`public static void func(int x) {`	Return type and access modifier required
`return x`	`return x;`	Semicolon
`# comment`	`// comment`	
`None`	`null`	
`True/False`	`true/false`	Lowercase in Java
`len(list)`	`list.size()`	Method call, not function
`list.append(x)`	`list.add(x)`	
`dict["key"]`	`map.get("key")`	
`try/except`	`try/catch`	
`import math`	`import java.lang.Math;` (or use `Math.` directly)	

Thought experiments for bridging:

1. *"In Python you write `x = 5` and Python figures out the type. In Java you write `int x = 5` and declare the type explicitly. Which approach do you prefer after using both? Which one would you choose for a large team project? For a quick experiment?"*

2. *"Python lists can hold any mixture of types: `[1, 'hello', True, 3.14]`. Java's `ArrayList<String>` can only hold Strings. What bugs does Python's flexibility enable? What bugs does it prevent compared to Java?"*

3. *"Python functions are first-class objects — you can store them in variables, pass them as arguments. In Java (before lambdas), this was much more com-*

plex. Java added lambdas in 2014. What does it mean for a language to add a major feature 19 years after it was created?"

---

16. Assessment Strategies

Alternative Assessment Forms **Code review presentations:** The student explains their code out loud, as if presenting to a team. The teacher asks questions about design decisions. This tests understanding more authentically than multiple-choice questions.

Bug-hunting assessments: Provide a working program with 5–8 intentional bugs of different types. The student must find and fix them using only their screen reader and development tools. This directly tests real-world debugging skill.

Design walkthroughs: Give the student a problem and ask them to design the solution — class structure, methods, data structures — without writing any code. Assess the quality of the design, not the typing speed.

Pair programming sessions: Two students work on one computer. One dictates code; the other types. Roles swap every 10–15 minutes. This surfaces communication and reasoning skills that solo assessments miss.

---

17. Supporting Diverse Needs

For Braille Display Users Java's verbose syntax can be particularly burdensome on a braille display with limited width. Strategies:

- Keep lines under 79 characters (PEP 8 applies to style; the Java equivalent is generally under 100).
- Use shorter variable names for very long class hierarchies where the full name is unwieldy.
- Encourage use of IntelliJ IDEA, which has better braille display integration than VS Code in some configurations.

For Students New to Screen Readers If a student is still developing screen reader proficiency:

- Begin with JShell instead of file-based programming. Single-line interactions provide immediate feedback.
- Use pair work more extensively until independent navigation is fluent.
- Reduce the number of new concepts per session until screen reader fluency stabilizes.

---

18. Alternative and Supplementary Resources

Textbooks and Reading

Resource	Level	Format	Accessibility
Introduction to Programming Using Java (Eck, free online)	Beginner–Intermediate	Free HTML/PDF	HTML version excellent; comprehensive
Big Java: Early Objects (Horstmann)	Beginner	Print/ebook	Standard university text; clear explanations
Effective Java, 3rd Ed. (Bloch)	Intermediate–Advanced	Print/ebook	Essential for understanding Java's design philosophy
Head First Java (Sierra, Bates)	Beginner	Print	Very visual; but text portions accessible; engaging writing style
Java: The Complete Reference (Schildt)	Reference	Print/ebook	Encyclopedic; good for looking up specific features
Oracle Java Tutorials (docs.oracle.com)	Beginner–Advanced	Free HTML	Excellent HTML accessibility; definitive
Java Documentation (javadoc)	Reference	Free HTML	Standard library docs; fully screen-reader-accessible

Online Courses and Platforms

Platform	Course	Accessibility	Notes
edX / MITx 6.092	Introduction to Programming in Java	High	MIT course; video transcripts available
Coursera	Java Programming (Duke)	Medium	Free audit available
Codecademy	Learn Java	Partial	Some UI inaccessible; editor readable
MOOC.fi (University of Helsinki)	Java Programming MOOC	High	Text-based exercises; excellent accessibility
CodingBat Java	Java practice problems	High	Plain HTML; no complex UI

Platform	Course	Accessibility	Notes
Exercism Java Track	Structured exercises with mentoring	High	Terminal-based; excellent
LeetCode	Algorithm problems	Medium	Code editor accessible; UI moderate

MOOC.fi is especially recommended for blind students — the University of Helsinki's Java course has no-frills HTML exercises and very clear progression. It is one of the most accessible Java learning platforms available.

Video and Audio Resources

Resource	Description	Accessibility
CS50 AP (Harvard)	AP Computer Science A course using Java	Captions and transcripts
Derek Banas Java Tutorials (YouTube)	Concise concept-by-concept tutorials	Captions available
Cave of Programming Java (YouTube/website)	Beginner Java series	Captions; text summaries
thenewboston Java (YouTube)	Large beginner series	Captions available
Coding with John (YouTube)	Clear explanations of Java concepts	Good pacing for beginners

---

19. Quick Reference Sheets

```
# Compile a single file
javac HelloWorld.java    # Creates HelloWorld.class

# Run compiled program
java HelloWorld          # No .class extension!

# Compile with output directory
javac -d bin src/HelloWorld.java

# Run JShell
jshell                   # Interactive REPL
```

```
/exit                    # Exit JShell

# Run tests (if using Maven)
mvn test

# Run tests (if using Gradle)
./gradlew test
```

Java Compilation and Running

Error Type Quick Reference

Error Type	Common Cause	First Check
`cannot find symbol`	Misspelled name or missing import	Spelling; check imports
`incompatible types`	Assigning wrong type or missing cast	Type of both sides of assignment
`';' expected`	Missing semicolon	End of the line indicated
`reached end of file while parsing`	Missing `}`	Count open braces vs close braces
`non-static method cannot be referenced from a static context`	Calling instance method from `main` without object	Need to create an object first
`NullPointerException`	Calling method on null reference	Which variable is null
`ArrayIndexOutOfBoundsException`	Index beyond array length	Print length and the index used
`ClassCastException`	Invalid cast	Check `instanceof` before casting
`StackOverflowError`	Infinite recursion	Verify base case
`NumberFormatException`	`parseInt()` on non-numeric string	Check input string before parsing

```
Scanner scanner = new Scanner(System.in);
```

```
String line   = scanner.nextLine();     // Entire line as String
String word   = scanner.next();         // Single token (whitespace-delimited)
int n         = scanner.nextInt();      // Integer
double d      = scanner.nextDouble();   // Double
boolean b     = scanner.nextBoolean();  // Boolean (true/false)

// Safe versions with type checking
if (scanner.hasNextInt()) {
    int n = scanner.nextInt();
}

// After nextInt(), consume leftover newline:
scanner.nextLine();  // ALWAYS do this before reading a String

// From file
Scanner fileScanner = new Scanner(new File("data.txt"));
// Same methods work; wrap in try-catch for FileNotFoundException

scanner.close();  // Always close when done
```

Scanner Method Reference

```

// Numbered list output
System.out.println("--- Contacts ---");
for (int i = 0; i < contacts.size(); i++) {
    System.out.printf("%d. %s%n", i + 1, contacts.get(i));
}
System.out.printf("Total: %d contacts%n", contacts.size());

// Menu
System.out.println("\n--- Main Menu ---");
System.out.println("1. Add contact");
System.out.println("2. View all contacts");
System.out.println("3. Search contacts");
System.out.println("4. Delete contact");
System.out.println("5. Save and quit");
System.out.println("-----------------");
System.out.print("Enter option (1-5): ");

// Confirmation messages
System.out.printf("Contact '%s' added successfully.%n", name);
System.out.printf("Found %d matching records.%n", count);
System.out.println("Data saved to contacts.json.");
System.out.println("Error: Name field cannot be empty. Please try again.");
```

Accessible Output Template

Unit 0: Setting Up Your Java Development Environment

Overview

This unit walks through installing Java and configuring VS Code for accessible Java development.

Lesson 0.1: Installing the Java Development Kit (JDK)

What Is the JDK? The **JDK (Java Development Kit)** includes everything you need to write, compile, and run Java programs:

- **JDK** — Tools for developers (compiler, debugger).
- **JVM (Java Virtual Machine)** — Runs compiled Java programs.
- **JRE (Java Runtime Environment)** — Libraries and runtime support.

Installing the JDK **Recommended:** Download Eclipse Temurin (OpenJDK) from `https://adoptium.net/`.

Choose **JDK 17 LTS** (Long Term Support) for stability.

Windows

1. Download the `.msi` installer.
2. **CRUCIAL STEP:** In the installer, ensure "Set JAVA_HOME variable" and "Add to PATH" are checked.
3. Activate "Install" and then "Finish".

macOS

1. Download the `.pkg` installer.
2. Follow the standard macOS installation steps.
3. The JDK is automatically added to your PATH.

```
sudo apt update
sudo apt install openjdk-17-jdk
```

Linux (Debian/Ubuntu)

Verify Installation Open a terminal and run:

```
java --version
javac --version
```

You should hear the version number (e.g., "openjdk 17.0.x").

Skills Checklist

☐ I have installed JDK 17 or later.
☐ Running `java --version` in the terminal shows the version number.
☐ Running `javac --version` in the terminal shows the compiler version.

Lesson 0.2: Installing VS Code and the Java Extension Pack

1. Install VS Code from https://code.visualstudio.com/.
2. Open VS Code and press `Ctrl + Shift + X` to open Extensions.
3. Search for **"Extension Pack for Java"** by Microsoft. Install it. This installs: Language Support for Java, Debugger for Java, Java Test Runner, and Maven support.

Configure Screen Reader Mode

- Press `Ctrl + Shift + P` -> type "Toggle Screen Reader Optimized" -> press Enter.

VS Code Keyboard Shortcuts for Java

Action	Shortcut
Run Java file	`Ctrl + F5`
Debug Java file	`F5`
Command Palette	`Ctrl + Shift + P`
Problems View	`Ctrl + Shift + M`
Next Error	`F8`
Go to Definition	`F12`
Format Code	`Shift + Alt + F`

Skills Checklist

☐ VS Code is installed with the Extension Pack for Java.
☐ I can toggle Screen Reader Optimized Mode.
☐ I can navigate errors using `F8`.

Lesson 0.3: Your First Java Project

Creating a Project

1. Press `Ctrl + Shift + P` -> "Java: Create Java Project".
2. Select "No build tools".
3. Choose a folder and name the project `HelloWorld`.
4. VS Code will create a `src` folder with a sample file.

Writing Hello World Create a file `HelloWorld.java` in the `src` folder:

```
public class HelloWorld {
    public static void main(String[] args) {
        System.out.println("Hello, World!");
    }
}
```

Running the Program

- Press `Ctrl + F5` to run without debugging.
- Or open the terminal (‘Ctrl + “) and type:

```
javac HelloWorld.java
java HelloWorld
```

Key Differences from Python

Python	Java
No required structure	Every program needs a `class`
No semicolons	Statements end with `;`
Indentation defines blocks	Curly braces `{}` define blocks
`print()`	`System.out.println()`
Interpreted	Compiled (`.java` -> `.class` -> run)
Dynamic typing	Static typing (declare variable types)

Skills Checklist

☐ I can create a new Java project in VS Code.
☐ I can write and run a Hello World program.
☐ I understand the basic structure of a Java program.

Unit 1: Foundations of Computing and Getting Started with Java

Overview

This unit introduces computer science concepts and the fundamentals of Java programming.

Objectives:

- Understand hardware vs. software.
- Write and run basic Java programs.
- Use variables and data types.

Lesson 1.1: Welcome to Computer Science

(See Python Unit 1.1 for conceptual content — hardware, software, algorithms, and problem-solving are language-agnostic.)

Java-Specific Notes Java was created by James Gosling at Sun Microsystems in 1995. Key principles:

- **"Write Once, Run Anywhere"** — Java programs compile to bytecode that runs on any JVM.
- **Strongly typed** — Every variable must have a declared type.
- **Object-Oriented** — Everything lives inside classes.

Lesson 1.2: Your First Java Program

```
// This is a comment
public class HelloWorld {                    // Class declaration
    public static void main(String[] args) { // Main method --- program entry point
        System.out.println("Hello, World!"); // Print a line to the console
    }
}
```

Anatomy of a Java Program **Key rules:**

- The file name **must match** the class name: `HelloWorld.java`.
- Every Java program starts execution from `public static void main(String[] args)`.
- Statements end with a semicolon `;`.
- Blocks of code are surrounded by curly braces `{}`.

```
System.out.println("This prints with a newline at the end.");
System.out.print("This prints without a newline.");
System.out.printf("My name is %s and I am %d years old.%n", "Alice", 15);
```

Output Methods

Examples **Example 1.2.1: Simple Greeting**

```
public class SimpleGreeting {
    public static void main(String[] args) {
        System.out.println("Hello, World!");
        System.out.println("Welcome to Java programming!");
    }
}
```

Example 1.2.2: Using printf for Formatted Output

```
public class FormattedOutput {
    public static void main(String[] args) {
        System.out.printf("Pi is approximately %.2f%n", 3.14159);
        System.out.printf("%-15s %5d%n", "Apples:", 42);
    }
}
```

Skills Checklist

☐ I can write a basic Java program with a `main` method.
☐ I can use `System.out.println()` to display output.
☐ I understand that Java files must match their class name.

Lesson 1.3: Variables and Primitive Data Types

Primitive Data Types

Type	Description	Size	Example
`int`	Whole numbers	32-bit	`int age = 25;`
`long`	Large whole numbers	64-bit	`long population = 8000000000L;`
`double`	Decimal numbers	64-bit	`double pi = 3.14159;`
`float`	Decimal (less precise)	32-bit	`float price = 9.99f;`
`boolean`	True or False	1-bit	`boolean isRaining = false;`
`char`	Single character	16-bit	`char grade = 'A';`

```
String name = "Alice";
String message = "Hello, " + name + "!";  // Concatenation
```

String (Reference Type)

```
// Declare and assign in one line
int score = 100;
double price = 4.99;
boolean isActive = true;
```

```
String studentName = "Charlie";

// Declare first, assign later
int total;
total = 50 + 25;
```

Variable Declaration and Assignment

```
final double PI = 3.14159265358979;
final int MAX_SIZE = 100;
```

Constants

```
// Widening (automatic --- safe)
int intValue = 42;
double doubleValue = intValue;   // 42 -> 42.0

// Narrowing (explicit cast --- may lose data)
double d = 9.99;
int i = (int) d;                 // 9.99 -> 9

// String to number
int parsed = Integer.parseInt("123");
double parsedD = Double.parseDouble("3.14");

// Number to String
String s = String.valueOf(42);
String s2 = Integer.toString(42);
```

Type Conversion

Arithmetic Operators

Operator	Description	Example
`+`	Addition	`5 + 3` = 8
`-`	Subtraction	`10 - 4` = 6
`*`	Multiplication	`7 * 6` = 42
`/`	Division	`15 / 4` = 3 (integer division!)
`%`	Modulo	`10 % 3` = 1
`++`	Increment	`count++`
`--`	Decrement	`count--`

Important: Integer division in Java truncates: `15 / 4` = `3`, not `3.75`. To get a decimal result, use `double`: `15.0 / 4` = `3.75`.

Examples **Example 1.3.1: Variables and Arithmetic**

```
public class VariablesDemo {
    public static void main(String[] args) {
        int appleCount = 5;
        int orangeCount = 3;
        int totalFruit = appleCount + orangeCount;
        System.out.println("You have " + totalFruit + " fruits in total.");

        double pricePerItem = 2.50;
        int quantity = 4;
        double totalCost = pricePerItem * quantity;
        System.out.println("The total cost is: " + totalCost);

        // Integer division
        int a = 10, b = 3;
        System.out.println("10 / 3 = " + (a / b));       // 3
        System.out.println("10.0 / 3 = " + (10.0 / b));  // 3.3333...
        System.out.println("10 % 3 = " + (a % b));        // 1
    }
}
```

Activities **Activity 1.3.1: Personal Information**

```
public class PersonalInfo {
    public static void main(String[] args) {
        String myName = "Jamie";
        int favoriteNumber = 7;
        double heightInMeters = 1.75;
        boolean likesPizza = true;

        System.out.println("My name is: " + myName);
        System.out.println("Favorite number: " + favoriteNumber);
        System.out.printf("Height: %.2f meters%n", heightInMeters);
        System.out.println("Likes pizza: " + likesPizza);
    }
}
```

Activity 1.3.2: Simple Math

```
public class SimpleMath {
    public static void main(String[] args) {
        int num1 = 15;
        int num2 = 7;

        System.out.println("Sum: " + (num1 + num2));
        System.out.println("Difference: " + (num1 - num2));
        System.out.println("Product: " + (num1 * num2));
        System.out.printf("Quotient: %.2f%n", (double) num1 / num2);
        System.out.println("Remainder: " + (num1 % num2));
```

```
    }
}
```

Skills Checklist

- ☐ I can declare variables with the correct Java types.
- ☐ I can perform basic arithmetic operations in Java.
- ☐ I understand the difference between integer and floating-point division.
- ☐ I can convert between types using casting and parse methods.
- ☐ I can use `final` to declare constants.

Unit 2: Writing Simple Programs and Input/Output

Overview

This unit covers getting user input, type conversions, and working with the Math library in Java.

Lesson 2.1: The Software Development Process

(See Python Unit 2.1 — the same design principles apply: analyze, design, implement, test.)

Lesson 2.2: Getting User Input with Scanner

The `Scanner` Class Java uses the `Scanner` class to read input from the user:

```
import java.util.Scanner;

public class UserInput {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);

        System.out.print("What is your name? ");
        String name = scanner.nextLine();

        System.out.print("How old are you? ");
        int age = scanner.nextInt();

        System.out.println("Hello, " + name + "!");
        System.out.println("Next year you will be " + (age + 1) + " years old.");

        scanner.close();
    }
}
```

Scanner Methods

Method	Reads	Example
`nextLine()`	Entire line as String	`String name = scanner.nextLine();`
`next()`	Single word (up to whitespace)	`String word = scanner.next();`
`nextInt()`	Integer	`int n = scanner.nextInt();`
`nextDouble()`	Double	`double d = scanner.nextDouble();`

Method	Reads	Example
`nextBoolean()`	Boolean	`boolean b = scanner.nextBoolean();`

Common Pitfall: *`nextInt()` + `nextLine()`* After calling `nextInt()`, there is a leftover newline character. Call `scanner.nextLine()` to consume it before reading another string:

```
int age = scanner.nextInt();
scanner.nextLine();            // Consume leftover newline
String name = scanner.nextLine();  // Now reads correctly
```

Examples **Example 2.2.1: Temperature Converter**

```
import java.util.Scanner;

public class TempConverter {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);

        System.out.print("Enter temperature in Fahrenheit: ");
        double fahrenheit = scanner.nextDouble();

        double celsius = (fahrenheit - 32) * 5.0 / 9.0;

        System.out.printf("%.1f°F is %.1f°C%n", fahrenheit, celsius);
        scanner.close();
    }
}
```

Example 2.2.2: Circle Calculator

```
import java.util.Scanner;

public class CircleCalculator {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);

        System.out.print("Enter the radius: ");
        double radius = scanner.nextDouble();

        double area = Math.PI * radius * radius;
        double circumference = 2 * Math.PI * radius;

        System.out.printf("Area: %.2f%n", area);
        System.out.printf("Circumference: %.2f%n", circumference);
        scanner.close();
    }
}
```

Skills Checklist

☐ I can import and create a `Scanner` object.
☐ I can read different types of input from the user.
☐ I can handle the `nextInt()`/`nextLine()` newline issue.

Lesson 2.3: The Math Class

```
// Math class provides static methods --- no import needed
double squareRoot = Math.sqrt(25.0);     // 5.0
double power = Math.pow(2, 10);           // 1024.0
double absolute = Math.abs(-7.5);         // 7.5
int ceiling = (int) Math.ceil(4.2);       // 5
int floor = (int) Math.floor(4.8);        // 4
double rounded = Math.round(4.6);         // 5
double maxVal = Math.max(10, 20);         // 20
double minVal = Math.min(10, 20);         // 10
double piValue = Math.PI;                 // 3.14159...
double eValue = Math.E;                   // 2.71828...
double sinVal = Math.sin(Math.PI / 2);    // 1.0
double logVal = Math.log(Math.E);         // 1.0  (natural log)
double log10 = Math.log10(100);           // 2.0
```

Skills Checklist

☐ I can use common `Math` class methods.
☐ I can use `Math.PI` for circle calculations.

Unit 3: Control Flow — Decisions and Loops

Overview

This unit introduces conditional statements and loops in Java.

Objectives:

- Use `if`, `else if`, and `else` for decision-making.
- Use `for`, `while`, and `do-while` loops.
- Use `break` and `continue`.

Lesson 3.1: Boolean Expressions and `if` Statements

Comparison Operators

Operator	Meaning	Example
`==`	Equal to	`age == 18`
`!=`	Not equal to	`score != 0`
`<`	Less than	`temp < 0`
`>`	Greater than	`score > 90`
`<=`	Less than or equal	`age <= 17`
`>=`	Greater than or equal	`score >= 60`

Logical Operators

Operator	Meaning	Example
`&&`	AND (both true)	`age >= 18 && hasId == true`
`||`	OR (at least one true)	`day.equals("Sat") || day.equals("Sun")`
`!`	NOT (negates)	`!isRaining`

```
int score = 85;

if (score >= 90) {
    System.out.println("Grade: A");
} else if (score >= 80) {
    System.out.println("Grade: B");
} else if (score >= 70) {
    System.out.println("Grade: C");
} else {
    System.out.println("Grade: F");
}
```

if-else if-else

String Comparison **Important:** Never use `==` to compare Strings. Use `.equals()`:

```
String color = "blue";
if (color.equals("blue")) {
    System.out.println("It's blue!");
}
if (color.equalsIgnoreCase("BLUE")) {
    System.out.println("Case-insensitive match!");
}
```

```
int day = 3;
switch (day) {
    case 1:
        System.out.println("Monday");
        break;
    case 2:
        System.out.println("Tuesday");
        break;
    case 3:
        System.out.println("Wednesday");
        break;
    default:
        System.out.println("Other day");
}
```

`switch` Statement

Examples **Example 3.1.1: Age Check**

```
import java.util.Scanner;

public class AgeCheck {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);
        System.out.print("Enter your age: ");
        int age = scanner.nextInt();

        if (age >= 18) {
            System.out.println("You are an adult.");
        } else {
            System.out.println("You are a minor.");
        }
        scanner.close();
    }
}
```

Example 3.1.2: Grade Calculator

```
import java.util.Scanner;

public class GradeCalculator {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);
        System.out.print("Enter your score (0-100): ");
        int score = scanner.nextInt();

        String grade;
        if (score >= 90) {
            grade = "A";
```

```
        } else if (score >= 80) {
            grade = "B";
        } else if (score >= 70) {
            grade = "C";
        } else if (score >= 60) {
            grade = "D";
        } else {
            grade = "F";
        }

        System.out.println("Your grade: " + grade);
        scanner.close();
    }
}
```

Skills Checklist

- ☐ I can write `if`, `else if`, and `else` statements in Java.
- ☐ I can use `&&`, `||`, and `!` to combine conditions.
- ☐ I use `.equals()` (not `==`) to compare Strings.
- ☐ I can write a `switch` statement.

Lesson 3.2: `for` Loops

```
// Basic for loop
for (int i = 0; i < 5; i++) {
    System.out.println("i = " + i);
}

// Counting down
for (int i = 10; i > 0; i--) {
    System.out.println(i);
}
System.out.println("Blast off!");

// Stepping by 2
for (int i = 0; i <= 10; i += 2) {
    System.out.println(i);
}
```

```
String[] fruits = {"apple", "banana", "cherry"};
for (String fruit : fruits) {
    System.out.println("I like " + fruit + ".");
}
```

```
int[] numbers = {1, 2, 3, 4, 5};
int sum = 0;
for (int num : numbers) {
    sum += num;
}
System.out.println("Sum: " + sum);
```

Enhanced `for` Loop (for-each)

Examples **Example 3.2.1: Multiplication Table**

```
public class MultiplicationTable {
    public static void main(String[] args) {
        int n = 5;
        System.out.println("Multiplication table for " + n + ":");
        for (int i = 1; i <= 10; i++) {
            System.out.println(n + " x " + i + " = " + (n * i));
        }
    }
}
```

Skills Checklist

☐ I can write a basic `for` loop with initialization, condition, and update.
☐ I can use an enhanced `for` loop to iterate over an array.

Lesson 3.3: `while` and `do-while` Loops

```

int count = 1;
while (count <= 5) {
    System.out.println("Count: " + count);
    count++;
}
```

`while` Loop

`do-while` Loop The `do-while` loop runs at least once before checking the condition:

```
Scanner scanner = new Scanner(System.in);
String input;
do {
    System.out.print("Enter a positive number: ");
    int number = scanner.nextInt();
    System.out.println("You entered: " + number);
    System.out.print("Continue? (yes/no): ");
    input = scanner.next();
} while (input.equals("yes"));
```

```java
// break --- exit the loop entirely
for (int i = 0; i < 10; i++) {
    if (i == 5) break;
    System.out.println(i);   // Prints 0, 1, 2, 3, 4
}

// continue --- skip the current iteration
for (int i = 0; i < 10; i++) {
    if (i % 2 == 0) continue;
    System.out.println(i);   // Prints 1, 3, 5, 7, 9
}
```

`break` and `continue`

Skills Checklist

☐ I can write a `while` loop.
☐ I can write a `do-while` loop.
☐ I can use `break` to exit a loop early.
☐ I can use `continue` to skip an iteration.

Unit 4: Data Structures — Arrays and ArrayLists

Overview

This unit introduces Java's built-in array type and the flexible `ArrayList` class.

Lesson 4.1: Arrays

```java
// Declare and allocate
int[] numbers = new int[5];            // [0, 0, 0, 0, 0]

// Declare and initialize with values
String[] colors = {"red", "blue", "green", "yellow"};

// Access elements (zero-based index)
System.out.println(colors[0]);         // "red"
System.out.println(colors[colors.length - 1]);  // "yellow"

// Modify an element
colors[1] = "purple";

// Loop through array
```

```
for (int i = 0; i < numbers.length; i++) {
    numbers[i] = i * 2;
    System.out.println("numbers[" + i + "] = " + numbers[i]);
}
```

Declaring and Initializing Arrays

```
import java.util.Arrays;

int[] data = {5, 2, 9, 1, 7, 3};

// Sort
Arrays.sort(data);
System.out.println(Arrays.toString(data));   // [1, 2, 3, 5, 7, 9]

// Search (array must be sorted first)
int index = Arrays.binarySearch(data, 7);
System.out.println("Found 7 at index: " + index);

// Copy
int[] copy = Arrays.copyOf(data, data.length);
int[] partial = Arrays.copyOfRange(data, 1, 4);  // [2, 3, 5]
```

Common Array Operations

Skills Checklist

☐ I can declare and initialize an array in Java.
☐ I can access and modify array elements by index.
☐ I can iterate over an array using a `for` loop.
☐ I can use `Arrays.sort()` and `Arrays.toString()`.

Lesson 4.2: ArrayLists

`ArrayList` is a resizable array that can grow or shrink dynamically.

```
import java.util.ArrayList;

ArrayList<String> shoppingList = new ArrayList<>();

// Add items
shoppingList.add("milk");
shoppingList.add("bread");
shoppingList.add("eggs");
shoppingList.add(1, "butter");   // Insert at index 1
```

```java
// Access items
System.out.println(shoppingList.get(0));          // "milk"
System.out.println(shoppingList.size());           // 4

// Remove items
shoppingList.remove("bread");                      // Remove by value
shoppingList.remove(0);                            // Remove by index

// Check membership
boolean hasMilk = shoppingList.contains("milk");

// Loop through
for (String item : shoppingList) {
    System.out.println("- " + item);
}
```

ArrayList vs. Array

Feature	`Array`	`ArrayList`
Size	Fixed	Dynamic
Type	Primitive or Object	Objects only
Syntax	`int[]`	`ArrayList<Integer>`
`.length` / `.size()`	`.length`	`.size()`
Performance	Slightly faster	More flexible

Skills Checklist

☐ I can create an `ArrayList` with a generic type.
☐ I can add, access, remove, and search items in an `ArrayList`.
☐ I can explain when to use an array vs. an `ArrayList`.

Lesson 4.3: Multidimensional Arrays

```java
// 2D array (matrix)
int[][] matrix = {
    {1, 2, 3},
    {4, 5, 6},
    {7, 8, 9}
};

// Access element at row 1, column 2
System.out.println(matrix[1][2]);   // 6

// Loop through 2D array
for (int row = 0; row < matrix.length; row++) {
```

```
    for (int col = 0; col < matrix[row].length; col++) {
        System.out.print(matrix[row][col] + " ");
    }
    System.out.println();
}
```

Skills Checklist

☐ I can declare and initialize a 2D array.
☐ I can access elements using row and column indices.
☐ I can iterate through a 2D array using nested loops.

Unit 5: Strings and String Processing

Overview

Java's `String` class is rich with methods for text manipulation.

Lesson 5.1: Working with Strings

```
String name = "Alice Smith";

// Length
System.out.println(name.length());           // 11

// Accessing characters
System.out.println(name.charAt(0));          // 'A'
System.out.println(name.charAt(name.length() - 1));  // 'h'

// Substrings
System.out.println(name.substring(6));       // "Smith"
System.out.println(name.substring(0, 5));    // "Alice"

// Case conversion
System.out.println(name.toUpperCase());      // "ALICE SMITH"
System.out.println(name.toLowerCase());      // "alice smith"

// Trimming whitespace
String padded = "  hello  ";
System.out.println(padded.trim());           // "hello"
System.out.println(padded.strip());          // "hello" (better Unicode support)

// Searching
System.out.println(name.indexOf("Smith"));   // 6
System.out.println(name.contains("Alice"));  // true
System.out.println(name.startsWith("Ali"));  // true
System.out.println(name.endsWith("ith"));    // true

// Replacing
String result = name.replace("Alice", "Bob");   // "Bob Smith"
String clean = "a,b,c".replace(",", " ");       // "a b c"

// Splitting
String sentence = "the quick brown fox";
String[] words = sentence.split(" ");
for (String word : words) {
    System.out.println(word);
}

// Checking equality
```

```
String s1 = "hello";
String s2 = "hello";
System.out.println(s1.equals(s2));           // true (correct)
System.out.println(s1.equalsIgnoreCase("HELLO")); // true

// Converting other types to String
String fromInt = String.valueOf(42);
String fromDouble = String.valueOf(3.14);
String formatted = String.format("Name: %-10s Age: %3d", "Alice", 15);
```

String Basics

`StringBuilder` for Efficient Concatenation Use `StringBuilder` when building strings in a loop (more efficient than `+`):

```
StringBuilder sb = new StringBuilder();
for (int i = 1; i <= 5; i++) {
    sb.append("Item ").append(i).append("\n");
}
System.out.println(sb.toString());
```

Examples **Example 5.1.1: String Analysis**

```
import java.util.Scanner;

public class StringAnalysis {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);
        System.out.print("Enter a sentence: ");
        String sentence = scanner.nextLine();

        String[] words = sentence.trim().split("\\s+");

        System.out.println("Original: " + sentence);
        System.out.println("Character count: " + sentence.length());
        System.out.println("Word count: " + words.length);
        System.out.println("Uppercase: " + sentence.toUpperCase());
        System.out.println("Lowercase: " + sentence.toLowerCase());

        scanner.close();
    }
}
```

Example 5.1.2: Word Frequency Counter

```
import java.util.HashMap;
import java.util.Map;

public class WordFrequency {
    public static void main(String[] args) {
```

```
        String text = "the quick brown fox jumps over the lazy dog the quick brown fox";
        String[] words = text.toLowerCase().split("\\s+");

        Map<String, Integer> frequency = new HashMap<>();
        for (String word : words) {
            frequency.put(word, frequency.getOrDefault(word, 0) + 1);
        }

        System.out.println("Word Frequencies:");
        for (Map.Entry<String, Integer> entry : frequency.entrySet()) {
            System.out.println("'" + entry.getKey() + "': " + entry.getValue());
        }
    }
}
```

Skills Checklist

☐ I can use common `String` methods in Java.
☐ I can compare strings using `.equals()`.
☐ I can split a string into an array of words.
☐ I can use `StringBuilder` for efficient string building.

Unit 6: Methods (Functions)

Overview

Java organizes reusable code into **methods** (the Java equivalent of Python functions).

Lesson 6.1: Defining and Calling Methods

```
accessModifier returnType methodName(paramType paramName, ...) {
    // method body
    return value;  // if returnType is not void
}
```

Method Syntax

Examples **Example 6.1.1: Simple Greeting Method**

```
public class GreetingDemo {

    public static void greetUser() {
        System.out.println("Hello, Java programmer!");
    }
```

```
    public static void greetByName(String name) {
        System.out.println("Hello, " + name + "!");
    }

    public static void main(String[] args) {
        greetUser();
        greetByName("Alice");
        greetByName("Bob");
    }
}
```

Example 6.1.2: Return Values

```
public class Calculator {

    public static int addNumbers(int a, int b) {
        return a + b;
    }

    public static double calculateCircleArea(double radius) {
        return Math.PI * radius * radius;
    }

    public static String getGrade(int score) {
        if (score >= 90) return "A";
        else if (score >= 80) return "B";
        else if (score >= 70) return "C";
        else if (score >= 60) return "D";
        else return "F";
    }

    public static void main(String[] args) {
        int sum = addNumbers(5, 3);
        System.out.println("Sum: " + sum);

        double area = calculateCircleArea(5.0);
        System.out.printf("Circle area: %.2f%n", area);

        System.out.println("Grade for 85: " + getGrade(85));
    }
}
```

Method Overloading Java allows multiple methods with the same name but different parameters:

```
public static int add(int a, int b) { return a + b; }
public static double add(double a, double b) { return a + b; }
public static int add(int a, int b, int c) { return a + b + c; }
```

```java
public static int sum(int[] numbers) {
    int total = 0;
    for (int num : numbers) {
        total += num;
    }
    return total;
}

// Calling:
int[] data = {1, 2, 3, 4, 5};
System.out.println("Sum: " + sum(data));
```

Passing Arrays to Methods

```java
public static int sumAll(int... numbers) {
    int total = 0;
    for (int num : numbers) {
        total += num;
    }
    return total;
}

// Calling:
System.out.println(sumAll(1, 2, 3));          // 6
System.out.println(sumAll(10, 20, 30, 40));    // 100
```

Varargs (Variable Arguments)

Skills Checklist

☐ I can define a method with a return type and parameters.
☐ I can call a method and use its return value.
☐ I understand the difference between `void` and non-`void` methods.
☐ I can overload methods with different parameter lists.
☐ I can pass arrays to methods.

Unit 7: Classes and Object-Oriented Programming

Overview

Java is fundamentally object-oriented. This unit covers defining and using classes.

Lesson 7.1: Why Learn About Classes?

(See Python Unit 7.1 for conceptual content — OOP concepts are language-agnostic.)

Java is a **class-based** language — every Java program is a class. Understanding OOP is essential for Java.

Lesson 7.2: Creating and Using a Class

```java
public class Dog {
    // Instance variables (fields)
    private String name;
    private int age;

    // Constructor
    public Dog(String name, int age) {
        this.name = name;
        this.age = age;
    }

    // Methods
    public void sit() {
        System.out.println(name + " is now sitting.");
    }

    public void rollOver() {
        System.out.println(name + " rolled over!");
    }

    public void getOlder(int years) {
        this.age += years;
        System.out.println(name + " is now " + age + " years old.");
    }

    // Getters and Setters
    public String getName() { return name; }
    public int getAge() { return age; }
    public void setAge(int age) { this.age = age; }

    @Override
    public String toString() {
        return "Dog{name='" + name + "', age=" + age + "}";
    }
}
```

Class Syntax

```java
public class DogDemo {
    public static void main(String[] args) {
        Dog myDog = new Dog("Buddy", 3);
        Dog yourDog = new Dog("Lucy", 1);

        System.out.println("My dog's name is " + myDog.getName() + ".");
        myDog.sit();
        myDog.rollOver();
        myDog.getOlder(2);

        System.out.println(myDog);
    }
}
```

Using the Class

Access Modifiers

Modifier	Access Level
`public`	Accessible from anywhere
`private`	Only within the same class
`protected`	Within the class and subclasses
(default)	Within the same package

Best practice: Make instance variables `private` and provide `public` getter/setter methods — this is **encapsulation**.

Skills Checklist

☐ I can define a Java class with fields, a constructor, and methods.
☐ I can create instances of a class using `new`.
☐ I understand the purpose of `this` in Java.
☐ I can write getter and setter methods.
☐ I understand access modifiers (`public`, `private`).

Lesson 7.3: Inheritance

```java
public class Vehicle {
    protected String make;
    protected String model;
    protected int year;

    public Vehicle(String make, String model, int year) {
        this.make = make;
```

```
        this.model = model;
        this.year = year;
    }

    public String getDescriptiveName() {
        return year + " " + make + " " + model;
    }

    public void readOdometer() {
        System.out.println("No odometer reading available.");
    }
}

public class Car extends Vehicle {
    private int odometer;

    public Car(String make, String model, int year) {
        super(make, model, year);   // Call parent constructor
        this.odometer = 0;
    }

    @Override
    public void readOdometer() {
        System.out.println("This car has " + odometer + " miles on it.");
    }

    public void updateOdometer(int miles) {
        if (miles >= odometer) {
            odometer = miles;
        } else {
            System.out.println("Error: Cannot roll back odometer.");
        }
    }
}

public class ElectricCar extends Car {
    private int batterySize;

    public ElectricCar(String make, String model, int year) {
        super(make, model, year);
        this.batterySize = 75;
    }

    public void describeBattery() {
        System.out.println("This car has a " + batterySize + "-kWh battery.");
    }
}
```

```java
// Abstract class --- cannot be instantiated
public abstract class Shape {
    public abstract double area();      // Abstract method --- no body
    public abstract double perimeter(); // Must be implemented by subclasses

    public void printInfo() {           // Concrete method --- has body
        System.out.printf("Area: %.2f, Perimeter: %.2f%n", area(), perimeter());
    }
}

public class Circle extends Shape {
    private double radius;

    public Circle(double radius) { this.radius = radius; }

    @Override
    public double area() { return Math.PI * radius * radius; }

    @Override
    public double perimeter() { return 2 * Math.PI * radius; }
}

// Interface --- defines a contract
public interface Drawable {
    void draw();
    default void resize(double factor) {
        System.out.println("Resizing by factor " + factor);
    }
}
```

Abstract Classes and Interfaces

Skills Checklist

☐ I can create a subclass that extends a parent class.
☐ I can use `super()` to call the parent constructor.
☐ I can override methods using `@Override`.
☐ I understand abstract classes and interfaces.

Unit 8: Files and Exception Handling

Overview

This unit covers reading and writing files in Java and handling exceptions.

Lesson 8.1: Reading from Files

```
import java.io.*;
import java.util.*;

public class FileReaderDemo {
    public static void main(String[] args) {
        // Reading entire file using Scanner
        try (Scanner fileScanner = new Scanner(new File("my_notes.txt"))) {
            while (fileScanner.hasNextLine()) {
                String line = fileScanner.nextLine();
                System.out.println(line);
            }
        } catch (FileNotFoundException e) {
            System.out.println("Error: File not found - " + e.getMessage());
        }

        // Reading with BufferedReader (more efficient for large files)
        try (BufferedReader reader = new BufferedReader(new FileReader("my_notes.txt"))) {
            String line;
            while ((line = reader.readLine()) != null) {
                System.out.println(line);
            }
        } catch (IOException e) {
            System.out.println("Error reading file: " + e.getMessage());
        }

        // Reading all lines into a List
        try {
            List<String> lines = new java.nio.file.Files.readAllLines(
                java.nio.file.Paths.get("my_notes.txt")
            );
            for (String line : lines) {
                System.out.println(line);
            }
        } catch (IOException e) {
            System.out.println("Error: " + e.getMessage());
        }
    }
}
```

```
import java.nio.file.*;
import java.io.IOException;
import java.util.List;

public class ModernFileReader {
    public static void main(String[] args) {
        Path filePath = Paths.get("my_notes.txt");
```

```
        try {
            // Read all lines
            List<String> lines = Files.readAllLines(filePath);
            for (String line : lines) {
                System.out.println(line);
            }

            // Read entire file as one string
            String content = Files.readString(filePath);
            System.out.println(content);

        } catch (IOException e) {
            System.out.println("Error reading file: " + e.getMessage());
        }
    }
}
```

Simpler approach using `java.nio.file`

Lesson 8.2: Writing to Files

```
import java.io.*;
import java.nio.file.*;

public class FileWriterDemo {
    public static void main(String[] args) {
        // Writing with PrintWriter (creates new or overwrites)
        try (PrintWriter writer = new PrintWriter("output.txt")) {
            writer.println("Hello, Java file I/O!");
            writer.println("Writing is easy.");
            writer.printf("Value: %d%n", 42);
        } catch (FileNotFoundException e) {
            System.out.println("Error: Could not create file.");
        }

        // Appending with FileWriter(true = append mode)
        try (PrintWriter appender = new PrintWriter(new FileWriter("output.txt", true))) {
            appender.println("This line is appended.");
        } catch (IOException e) {
            System.out.println("Error appending to file.");
        }

        // Modern NIO approach
        try {
            String content = "Hello from NIO!\nSecond line.";
            Files.writeString(Paths.get("modern_output.txt"), content);

            // Append
```

```
            Files.writeString(Paths.get("modern_output.txt"),
                "\nAppended line.",
                StandardOpenOption.APPEND);
        } catch (IOException e) {
            System.out.println("Error: " + e.getMessage());
        }
    }
}
```

Lesson 8.3: Exception Handling

```
public class ExceptionHandling {
    public static void main(String[] args) {
        // Basic try-catch
        try {
            int result = 10 / 0;
        } catch (ArithmeticException e) {
            System.out.println("Error: " + e.getMessage());
        }

        // Multiple catch blocks
        try {
            String input = "abc";
            int number = Integer.parseInt(input);
        } catch (NumberFormatException e) {
            System.out.println("Invalid number format: " + e.getMessage());
        } catch (Exception e) {
            System.out.println("Unexpected error: " + e.getMessage());
        }

        // try-catch-finally
        PrintWriter writer = null;
        try {
            writer = new PrintWriter("data.txt");
            writer.println("Some data");
        } catch (FileNotFoundException e) {
            System.out.println("Could not create file.");
        } finally {
            if (writer != null) {
                writer.close();
                System.out.println("File closed.");
            }
        }

        // try-with-resources (automatically closes resources)
        try (PrintWriter pw = new PrintWriter("data2.txt")) {
            pw.println("Better approach!");
```

```
        } catch (FileNotFoundException e) {
            System.out.println("Could not create file.");
        }
    }
}
```

try-catch-finally

Common Java Exceptions

Exception	When it occurs
`ArithmeticException`	Division by zero
`NumberFormatException`	Invalid number conversion (e.g., `Integer.parseInt("abc")`)
`ArrayIndexOutOfBoundsException`	Array index out of range
`NullPointerException`	Calling method on null reference
`FileNotFoundException`	File doesn't exist
`IOException`	General I/O error
`ClassCastException`	Invalid type cast

```
public class InsufficientFundsException extends Exception {
    private double amount;

    public InsufficientFundsException(double amount) {
        super("Insufficient funds: need " + amount + " more.");
        this.amount = amount;
    }

    public double getAmount() { return amount; }
}

// Usage:
public void withdraw(double amount) throws InsufficientFundsException {
    if (amount > balance) {
        throw new InsufficientFundsException(amount - balance);
    }
    balance -= amount;
}
```

Custom Exceptions

Skills Checklist

☐ I can read from a file using `Scanner` or `BufferedReader`.
☐ I can write to a file using `PrintWriter`.

- ☐ I can use `try-catch-finally` to handle exceptions.
- ☐ I can use try-with-resources for automatic resource cleanup.
- ☐ I know common Java exception types.

Unit 9: Collections, Data Analysis, and Testing

Overview

This unit introduces Java's Collections Framework, basic data analysis, and unit testing with JUnit.

Lesson 9.1: Java Collections Framework

Key Interfaces and Classes

```
Collection
|--- List
|    |--- ArrayList
|    +--- LinkedList
+--- Set
     |--- HashSet
     +--- TreeSet

Map
|--- HashMap
+--- TreeMap
```

```
import java.util.HashMap;
import java.util.Map;

HashMap<String, Integer> inventory = new HashMap<>();

// Add entries
inventory.put("apples", 50);
inventory.put("bananas", 30);
inventory.put("oranges", 40);

// Access
System.out.println(inventory.get("apples"));           // 50
System.out.println(inventory.getOrDefault("grapes", 0)); // 0

// Check
System.out.println(inventory.containsKey("bananas"));   // true
System.out.println(inventory.containsValue(30));         // true

// Update
inventory.put("apples", 55);

// Remove
inventory.remove("bananas");
```

```
// Iterate
for (Map.Entry<String, Integer> entry : inventory.entrySet()) {
    System.out.println(entry.getKey() + ": " + entry.getValue());
}

// Keys only
for (String key : inventory.keySet()) {
    System.out.println(key);
}

// Values only
for (int value : inventory.values()) {
    System.out.println(value);
}

System.out.println("Size: " + inventory.size());
```

`HashMap` — Key-Value Storage

```
import java.util.HashSet;

HashSet<String> uniqueWords = new HashSet<>();
uniqueWords.add("apple");
uniqueWords.add("banana");
uniqueWords.add("apple");   // Duplicate --- ignored

System.out.println(uniqueWords.size());          // 2
System.out.println(uniqueWords.contains("banana")); // true
```

`HashSet` — Unique Elements

```
import java.util.TreeMap;

TreeMap<String, Integer> sorted = new TreeMap<>();
sorted.put("banana", 2);
sorted.put("apple", 1);
sorted.put("cherry", 3);

// Iterates in alphabetical key order
for (Map.Entry<String, Integer> e : sorted.entrySet()) {
    System.out.println(e.getKey() + ": " + e.getValue());
}
```

`TreeMap` — Sorted Keys

```
import java.util.*;

List<Integer> numbers = new ArrayList<>(Arrays.asList(5, 2, 9, 1, 7));
Collections.sort(numbers);
System.out.println(numbers);   // [1, 2, 5, 7, 9]

Collections.sort(numbers, Collections.reverseOrder());
System.out.println(numbers);   // [9, 7, 5, 2, 1]

int max = Collections.max(numbers);
int min = Collections.min(numbers);
```

Sorting with Collections

Lesson 9.2: Word Frequency Analysis

```
import java.util.*;

public class WordFrequency {
    public static Map<String, Integer> countWords(String text) {
        Map<String, Integer> frequency = new HashMap<>();
        String[] words = text.toLowerCase()
                             .replaceAll("[^a-z\\s]", "")
                             .trim()
                             .split("\\s+");

        for (String word : words) {
            frequency.put(word, frequency.getOrDefault(word, 0) + 1);
        }
        return frequency;
    }

    public static void main(String[] args) {
        String text = "the quick brown fox jumps over the lazy dog the quick brown fox";
        Map<String, Integer> freq = countWords(text);

        // Sort by frequency (most common first)
        List<Map.Entry<String, Integer>> entries = new ArrayList<>(freq.entrySet());
        entries.sort((a, b) -> b.getValue() - a.getValue());

        System.out.println("Word Frequencies:");
        for (Map.Entry<String, Integer> entry : entries) {
            System.out.printf("'%s': %d%n", entry.getKey(), entry.getValue());
        }
    }
}
```

Lesson 9.3: Working with CSV Files

```
import java.io.*;
import java.util.*;

public class CSVReader {
    public static void main(String[] args) {
        String filename = "students.csv";

        try (BufferedReader reader = new BufferedReader(new FileReader(filename))) {
            String headerLine = reader.readLine();
            String[] headers = headerLine.split(",");
            System.out.println("Columns: " + Arrays.toString(headers));
            System.out.println("---");

            String line;
            while ((line = reader.readLine()) != null) {
                String[] data = line.split(",");
                if (data.length == headers.length) {
                    String name = data[0].trim();
                    int age = Integer.parseInt(data[1].trim());
                    String grade = data[2].trim();
                    System.out.printf("Name: %-15s Age: %3d  Grade: %s%n",
                                      name, age, grade);
                }
            }
        } catch (FileNotFoundException e) {
            System.out.println("Error: File not found - " + filename);
        } catch (IOException e) {
            System.out.println("Error reading file: " + e.getMessage());
        } catch (NumberFormatException e) {
            System.out.println("Error: Invalid number format in CSV.");
        }
    }
}
```

Lesson 9.4: Unit Testing with JUnit

Adding JUnit to Your Project If using VS Code with Maven, add to `pom.xml`:

```
<dependency>
    <groupId>org.junit.jupiter</groupId>
    <artifactId>junit-jupiter</artifactId>
    <version>5.10.0</version>
    <scope>test</scope>
</dependency>
```

```java
// MyMath.java
public class MyMath {
    public static int add(int a, int b) { return a + b; }
    public static double circleArea(double radius) { return Math.PI * radius * radius; }
    public static String getGrade(int score) {
        if (score >= 90) return "A";
        if (score >= 80) return "B";
        if (score >= 70) return "C";
        return "F";
    }
}
```

```java
// TestMyMath.java
import org.junit.jupiter.api.*;
import static org.junit.jupiter.api.Assertions.*;

public class TestMyMath {

    @Test
    public void testAddPositiveNumbers() {
        assertEquals(8, MyMath.add(5, 3));
    }

    @Test
    public void testAddNegativeNumbers() {
        assertEquals(-8, MyMath.add(-5, -3));
    }

    @Test
    public void testCircleArea() {
        double expected = Math.PI * 25;
        assertEquals(expected, MyMath.circleArea(5.0), 0.0001);
    }

    @Test
    public void testGradeA() {
        assertEquals("A", MyMath.getGrade(95));
    }

    @Test
    public void testGradeF() {
        assertEquals("F", MyMath.getGrade(55));
    }
}
```

Writing Tests

Common JUnit Assertions

Method	Description
`assertEquals(expected, actual)`	Check equality
`assertTrue(condition)`	Check condition is true
`assertFalse(condition)`	Check condition is false
`assertNull(object)`	Check object is null
`assertNotNull(object)`	Check object is not null
`assertThrows(ExceptionClass, executable)`	Check exception is thrown

Skills Checklist

- ☐ I can use `HashMap` for key-value storage in Java.
- ☐ I can iterate through a map's entries, keys, and values.
- ☐ I can write unit tests using JUnit 5.
- ☐ I can use common assertion methods.

Unit 10: Project-Based Learning

Overview

This unit guides students through building text-based Java applications.

Lesson 10.1: Designing a Text-Based Adventure Game

```
import java.util.*;

public class Room {
    private String name;
    private String description;
    private Map<String, Room> exits;
    private List<String> items;

    public Room(String name, String description) {
        this.name = name;
        this.description = description;
        this.exits = new HashMap<>();
        this.items = new ArrayList<>();
    }

    public void addExit(String direction, Room room) {
        exits.put(direction.toLowerCase(), room);
    }
```

```
    public void addItem(String item) {
        items.add(item);
    }

    public Room getExit(String direction) {
        return exits.get(direction.toLowerCase());
    }

    public String getDescription() {
        StringBuilder sb = new StringBuilder();
        sb.append("--- ").append(name).append(" ---\n");
        sb.append(description).append("\n");
        if (!items.isEmpty()) {
            sb.append("You see: ").append(String.join(", ", items)).append("\n");
        }
        sb.append("Exits: ").append(String.join(", ", exits.keySet()));
        return sb.toString();
    }

    public String getName() { return name; }
    public List<String> getItems() { return items; }
}
```

```
import java.util.*;

public class Player {
    private String name;
    private Room currentRoom;
    private List<String> inventory;
    private int health;

    public Player(String name, Room startRoom) {
        this.name = name;
        this.currentRoom = startRoom;
        this.inventory = new ArrayList<>();
        this.health = 100;
    }

    public boolean move(String direction) {
        Room nextRoom = currentRoom.getExit(direction);
        if (nextRoom != null) {
            currentRoom = nextRoom;
            return true;
        }
        return false;
    }

    public boolean takeItem(String itemName) {
        List<String> roomItems = currentRoom.getItems();
        for (String item : roomItems) {
```

```
            if (item.equalsIgnoreCase(itemName)) {
                inventory.add(item);
                roomItems.remove(item);
                return true;
            }
        }
        return false;
    }

    public String getStatus() {
        return "Player: " + name + " | Health: " + health +
               " | Inventory: " + (inventory.isEmpty() ? "empty" : String.join(", ", inventory));
    }

    public Room getCurrentRoom() { return currentRoom; }
    public List<String> getInventory() { return inventory; }
}
```

```
import java.util.Scanner;

public class AdventureGame {
    public static void main(String[] args) {
        // Build the world
        Room cave = new Room("Dark Cave", "You are in a dimly lit cave.");
        Room tunnel = new Room("Stone Tunnel", "A narrow tunnel leads north and south.");
        Room clearing = new Room("Sunny Clearing", "A bright clearing with tall trees.");

        cave.addExit("north", tunnel);
        cave.addItem("torch");
        tunnel.addExit("south", cave);
        tunnel.addExit("north", clearing);
        clearing.addExit("south", tunnel);

        Player player = new Player("Hero", cave);
        Scanner scanner = new Scanner(System.in);

        System.out.println("=== Text Adventure ===");
        System.out.println("Commands: go [direction], take [item], inventory, look, quit");
        System.out.println();

        while (true) {
            System.out.println(player.getCurrentRoom().getDescription());
            System.out.println(player.getStatus());
            System.out.print("\n> ");

            String input = scanner.nextLine().trim().toLowerCase();
            String[] parts = input.split("\\s+", 2);
            String command = parts[0];
            String argument = parts.length > 1 ? parts[1] : "";
```

```
            switch (command) {
                case "go":
                    if (argument.isEmpty()) {
                        System.out.println("Go where?");
                    } else if (!player.move(argument)) {
                        System.out.println("You can't go that way.");
                    }
                    break;
                case "take":
                    if (player.takeItem(argument)) {
                        System.out.println("You picked up the " + argument + ".");
                    } else {
                        System.out.println("There's no " + argument + " here.");
                    }
                    break;
                case "inventory":
                    System.out.println("Inventory: " + player.getInventory());
                    break;
                case "look":
                    // Room description already shown at top of loop
                    break;
                case "quit":
                    System.out.println("Thanks for playing!");
                    scanner.close();
                    return;
                default:
                    System.out.println("Unknown command: " + command);
            }
            System.out.println();
        }
    }
}
```

Room and Player Structure

Lesson 10.2: Command-Line To-Do Manager

```
import java.util.*;
import java.io.*;
import com.google.gson.*;   // Add Gson library for JSON support

public class TodoManager {
    private List<Map<String, Object>> todos = new ArrayList<>();
    private final String FILE_PATH = "todos.json";

    public void addTask(String description) {
        Map<String, Object> task = new HashMap<>();
        task.put("description", description);
        task.put("completed", false);
```

```
        todos.add(task);
        System.out.println("Task added: " + description);
    }

    public void viewTasks() {
        if (todos.isEmpty()) {
            System.out.println("No tasks in your list.");
            return;
        }
        System.out.println("--- Your To-Do List ---");
        for (int i = 0; i < todos.size(); i++) {
            Map<String, Object> task = todos.get(i);
            String status = (Boolean) task.get("completed") ? "[X]" : "[ ]";
            System.out.printf("%d. %s %s%n", i + 1, status, task.get("description"));
        }
        System.out.println("-----------------------");
    }

    public void markComplete(int taskNumber) {
        int index = taskNumber - 1;
        if (index >= 0 && index < todos.size()) {
            todos.get(index).put("completed", true);
            System.out.println("Task marked complete: " + todos.get(index).get("description"));
        } else {
            System.out.println("Invalid task number.");
        }
    }

    public static void main(String[] args) {
        TodoManager manager = new TodoManager();
        Scanner scanner = new Scanner(System.in);

        System.out.println("=== To-Do Manager ===");

        while (true) {
            System.out.println("\n1. Add Task  2. View Tasks  3. Mark Complete  4. Quit");
            System.out.print("Choice: ");
            String choice = scanner.nextLine().trim();

            switch (choice) {
                case "1":
                    System.out.print("Task description: ");
                    manager.addTask(scanner.nextLine());
                    break;
                case "2":
                    manager.viewTasks();
                    break;
                case "3":
                    manager.viewTasks();
                    System.out.print("Task number: ");
                    try {
```

```
                    manager.markComplete(Integer.parseInt(scanner.nextLine()));
                } catch (NumberFormatException e) {
                    System.out.println("Invalid input.");
                }
                break;
            case "4":
                System.out.println("Goodbye!");
                scanner.close();
                return;
            default:
                System.out.println("Invalid choice.");
        }
    }
}
```

Unit 11: Capstone Project — Personal Data Manager in Java

Overview

Students build a fully functional command-line personal data manager in Java.

Project Options

- **Contact Manager:** name, phone, email, notes
- **Book Tracker:** title, author, genre, read status
- **Inventory System:** item name, quantity, location, notes
- **To-Do App:** description, due date, priority, status

Required Features

1. **Add item** — prompt for all attributes and save to list.
2. **View all items** — display all stored items clearly.
3. **Edit item** — select by number/name and modify.
4. **Delete item** — remove selected item.
5. **Search/filter** — find items by keyword.
6. **Persistence** — save/load to a JSON or CSV file.

Architecture Recommendations

```
src/
|--- model/
|   +--- Contact.java        (data model class)
|--- storage/
|   +--- DataManager.java    (file I/O)
|--- ui/
|   +--- ConsoleUI.java      (user interface)
+--- Main.java               (entry point)
```

Assessment Criteria

Functionality (60%)

☐ Add, view, edit, and delete items work correctly.
☐ Data persists between program runs.
☐ Program handles invalid input without crashing.

Code Quality (20%)

☐ Methods are small and have a single responsibility.
☐ Classes follow OOP principles (encapsulation).
☐ Meaningful names for all variables, methods, and classes.
☐ Appropriate comments and Javadoc.

Accessibility & UX (15%)

☐ Clear, descriptive prompts for all input.
☐ Explicit confirmation messages after every action.
☐ Consistent output format.
☐ Informative error messages.

Documentation (5%)

☐ `README.md` with setup and usage instructions.
☐ Reflection on challenges and solutions.